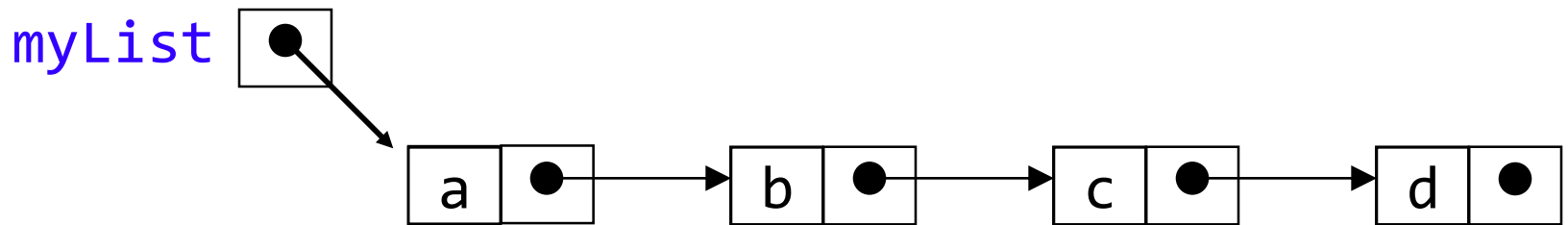


Anatomy of a linked list

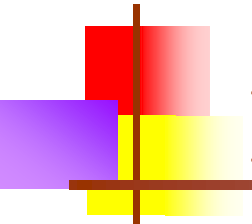
- A linked list consists of:
 - A sequence of **nodes**



Each node contains **values**
and **links** (pointer or reference) to some node

The last node contains a **null link**

The list may (or may not) have a **header**

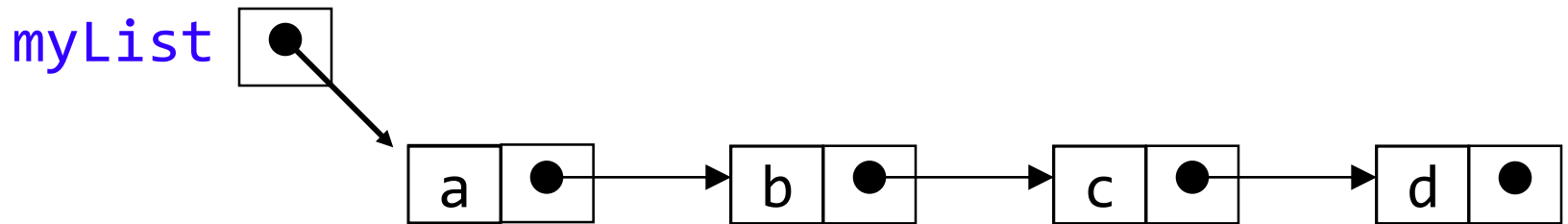


More terminology

- A node's **successor** is the next node in the sequence
 - The last node has no successor
- A node's **predecessor** is the previous node in the sequence
 - The first node has no predecessor
- A list's **length** is the number of elements in it
 - A list may be **empty** (contain no elements)

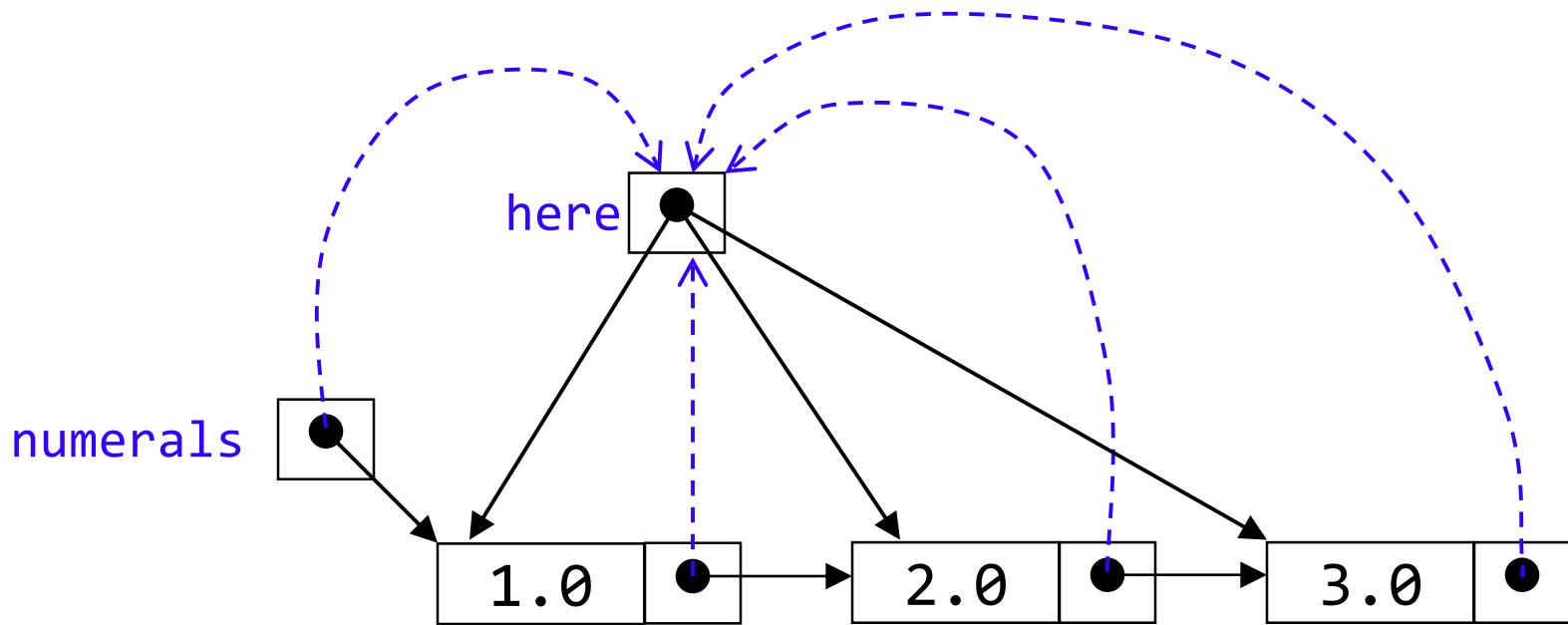
Singly-linked lists

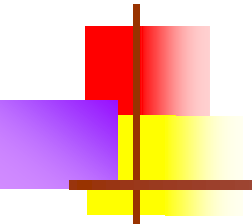
- Here is a **singly-linked list (SLL)**:



- Each node contains a value and a link to its successor (the last node has no successor)
- If header exists then its link points to the first node in the list (or contains the null link if the list is empty)

Traversing a SLL (animation)



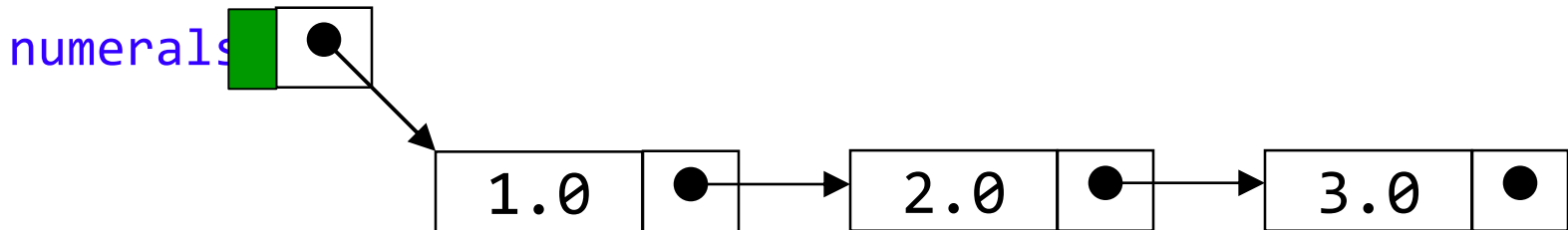


Inserting a node into a SLL

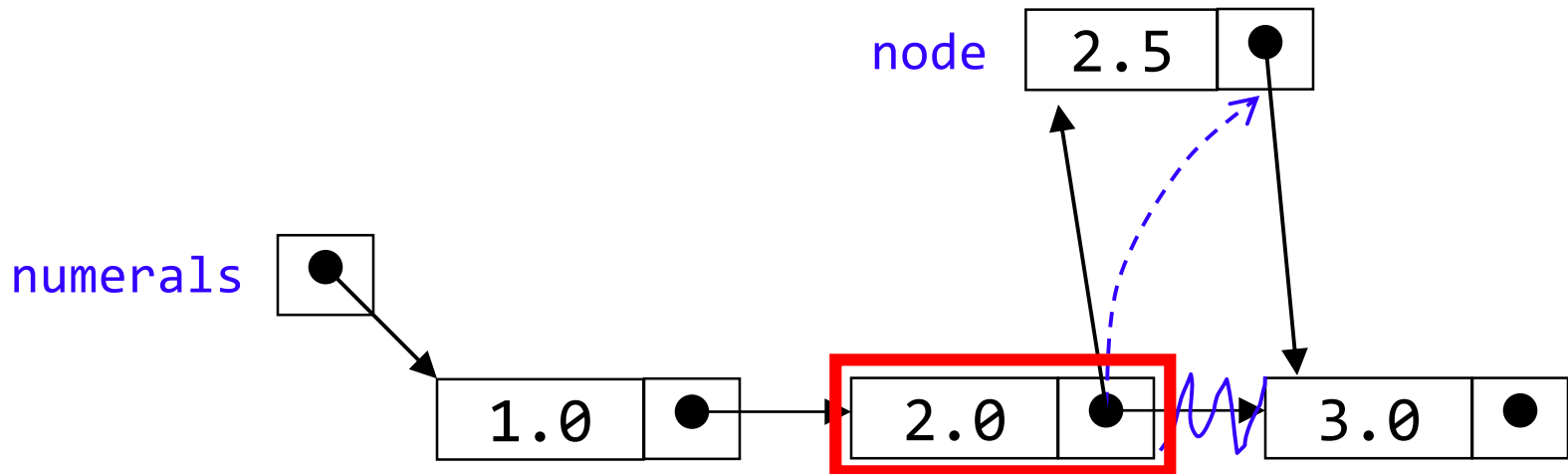
- There are many ways you might want to insert a new node into a list:
 - As the new first element
 - As the new last element
 - Before a given node (specified by a *reference*)
 - After a given node
 - Before a given value
 - After a given value
- All are possible, but differ in difficulty

Using a header node

- A header node (if exists) is just an initial node that exists at the front of a list, even when the list is empty
- The purpose is to keep the list (**including head node**) from being **null**, to point at the first element of the non-empty list (**excluding head node**), and may contain metadata.



Inserting after (animation)



Find the node you want to insert after

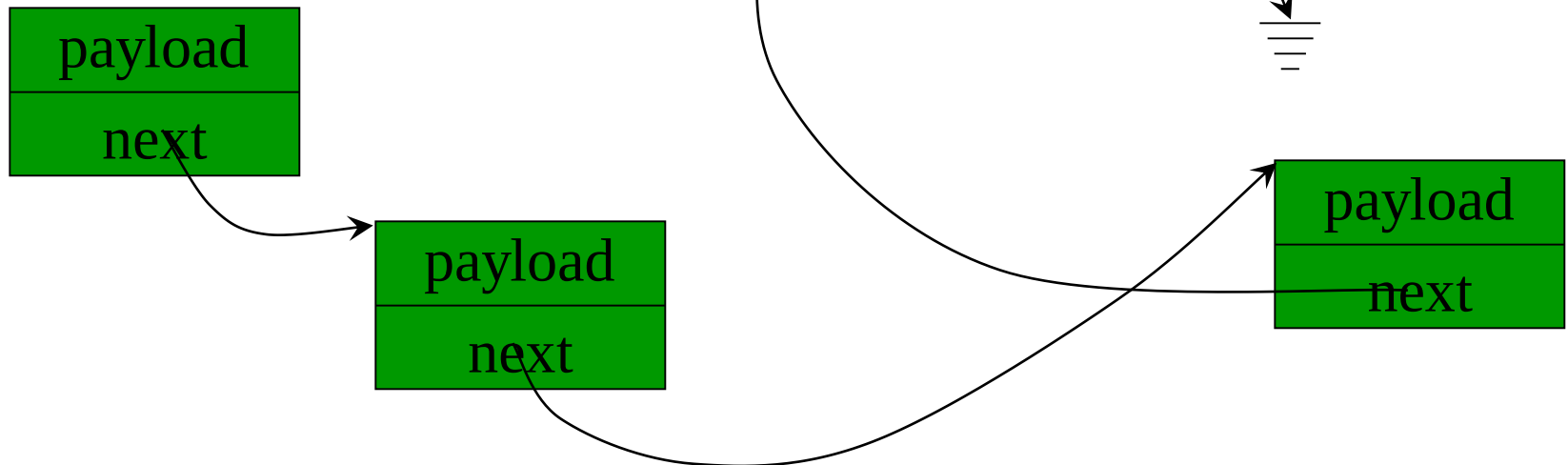
First, copy the link from the node that's already in the list

Then, change the link in the node that's already in the list

Linked List

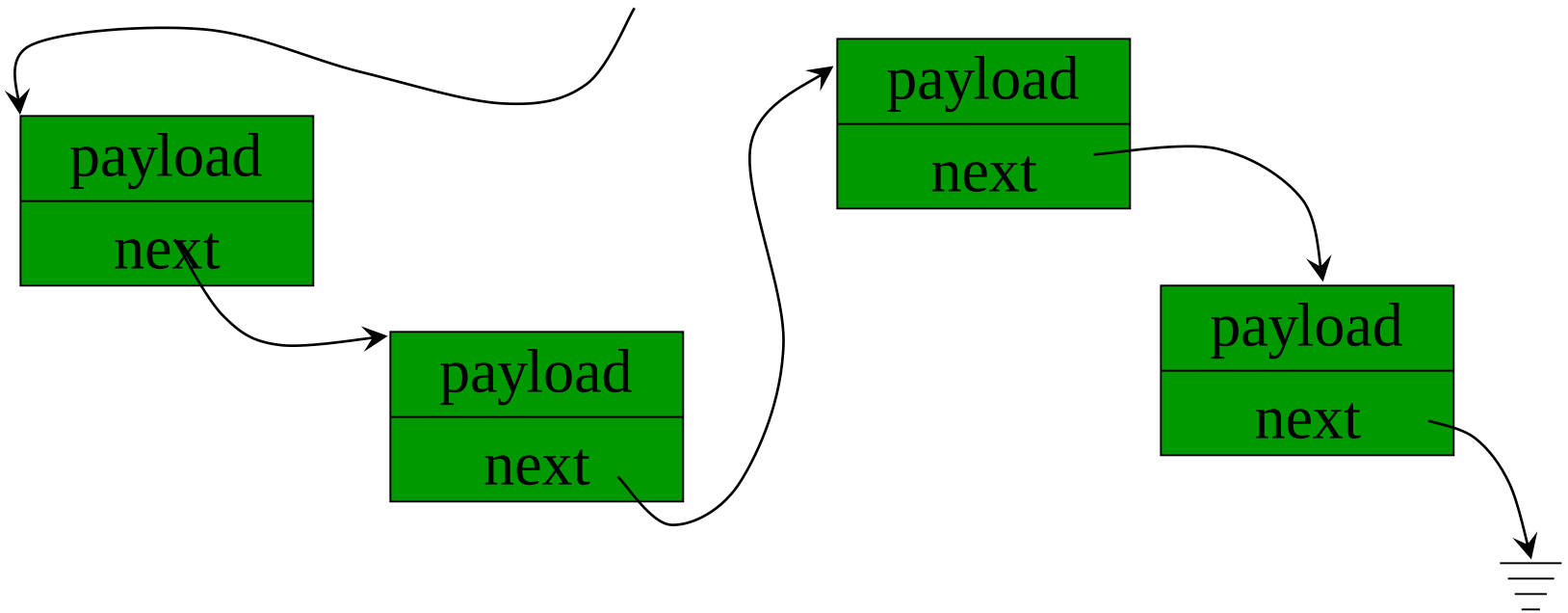
Note: payload may be multiple members.

```
struct listItem {  
    type payload;  
    struct listItem *next;  
};
```



Linked List (continued)

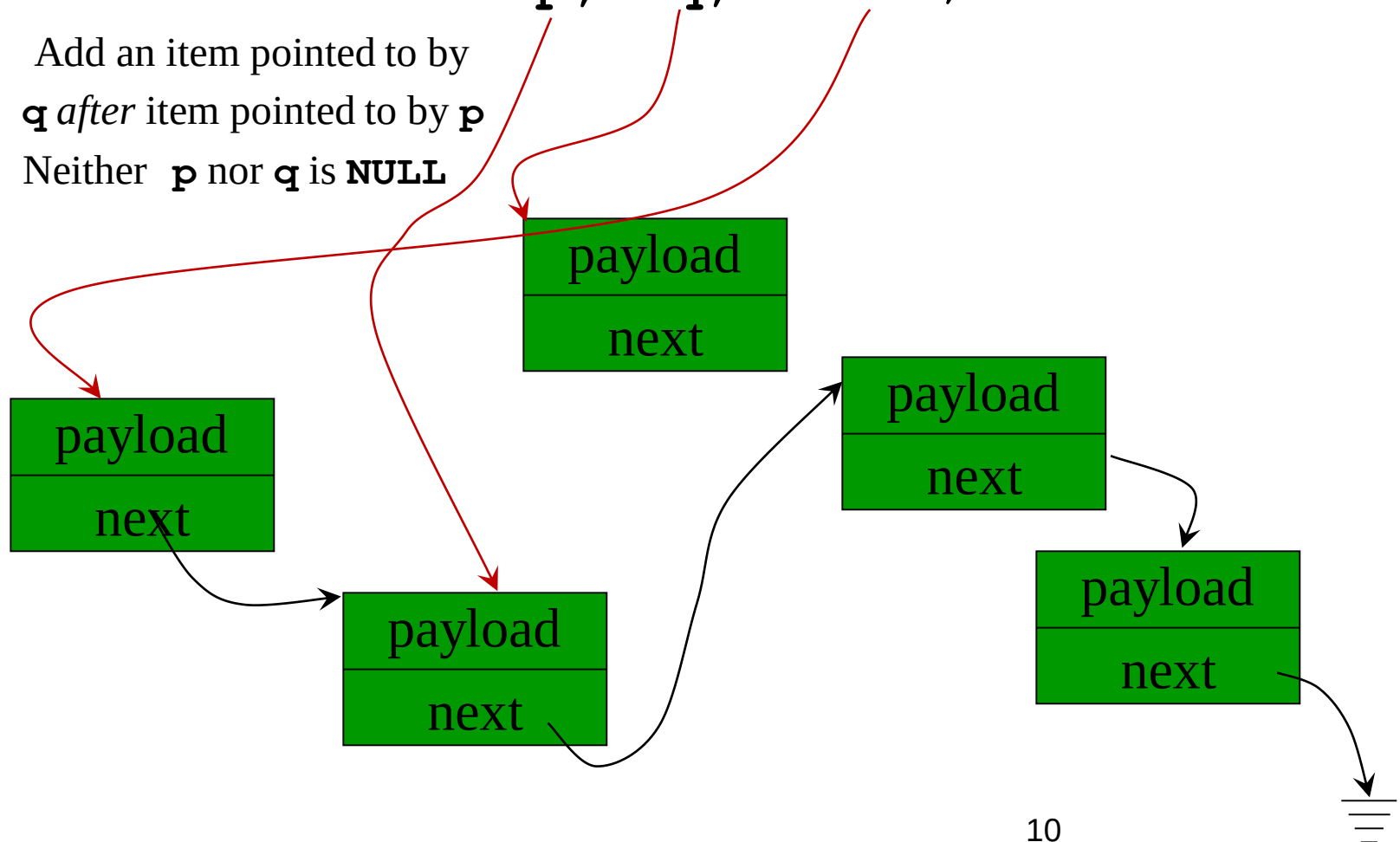
```
struct listItem {  
    type payload;  
    struct listItem *next;  
};  
struct listItem *head;
```



Adding an Item to a List

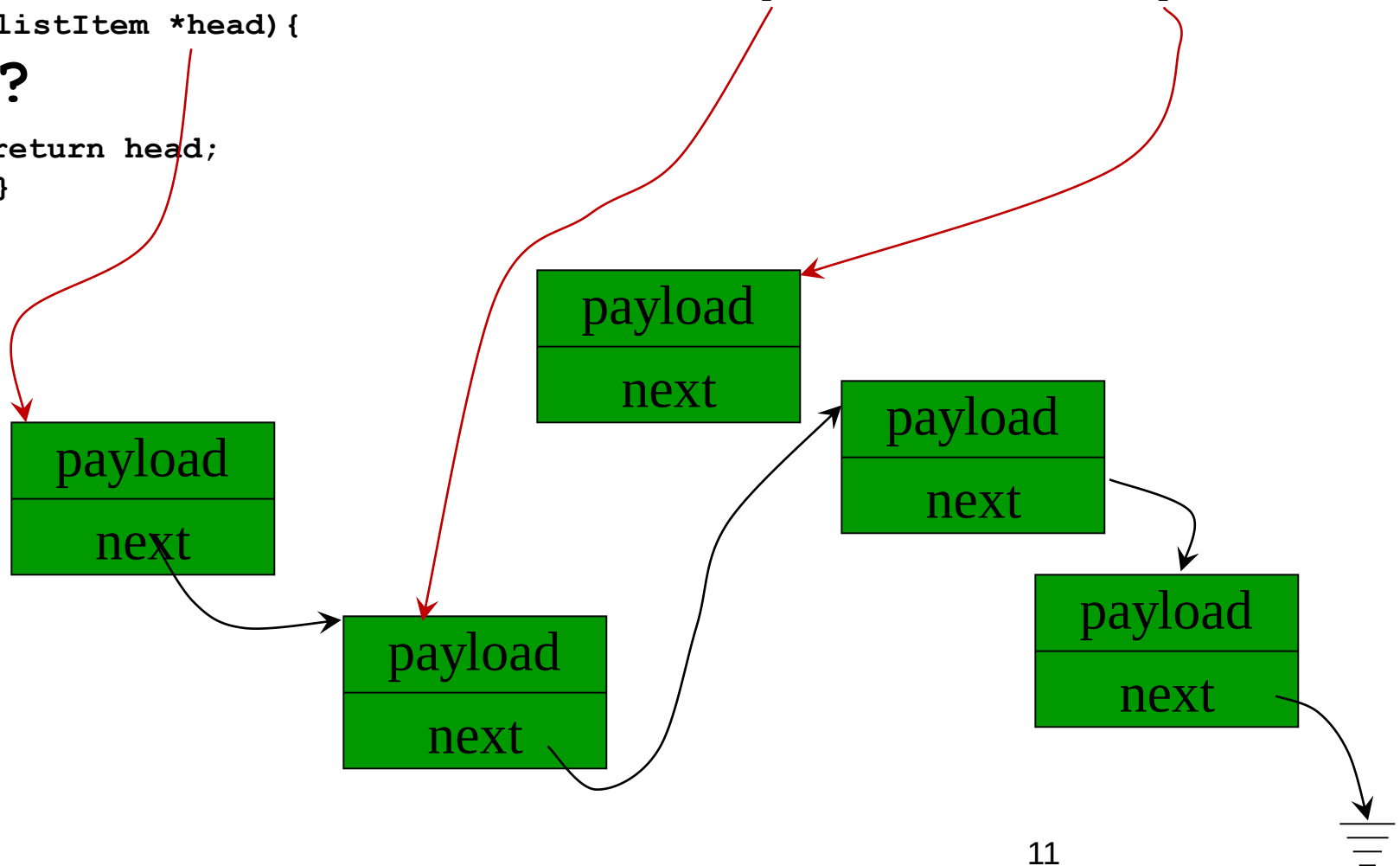
```
struct listItem *p, *q, *head;
```

- Add an item pointed to by **q** *after* item pointed to by **p**
- Neither **p** nor **q** is **NULL**



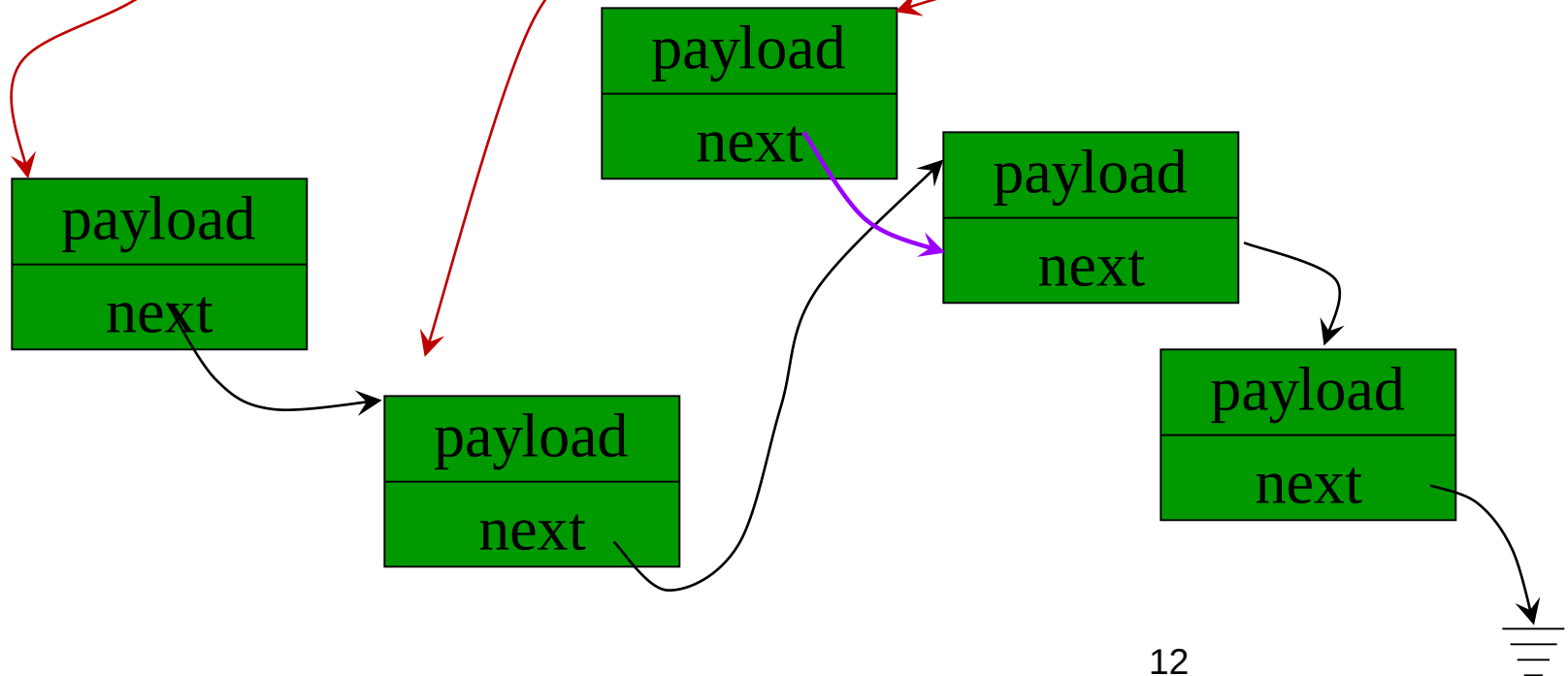
Adding an Item to a List

```
struct listItem *addAfter(struct listItem *p, struct listItem *q, struct  
    listItem *head){  
    ?  
    return head;  
}
```



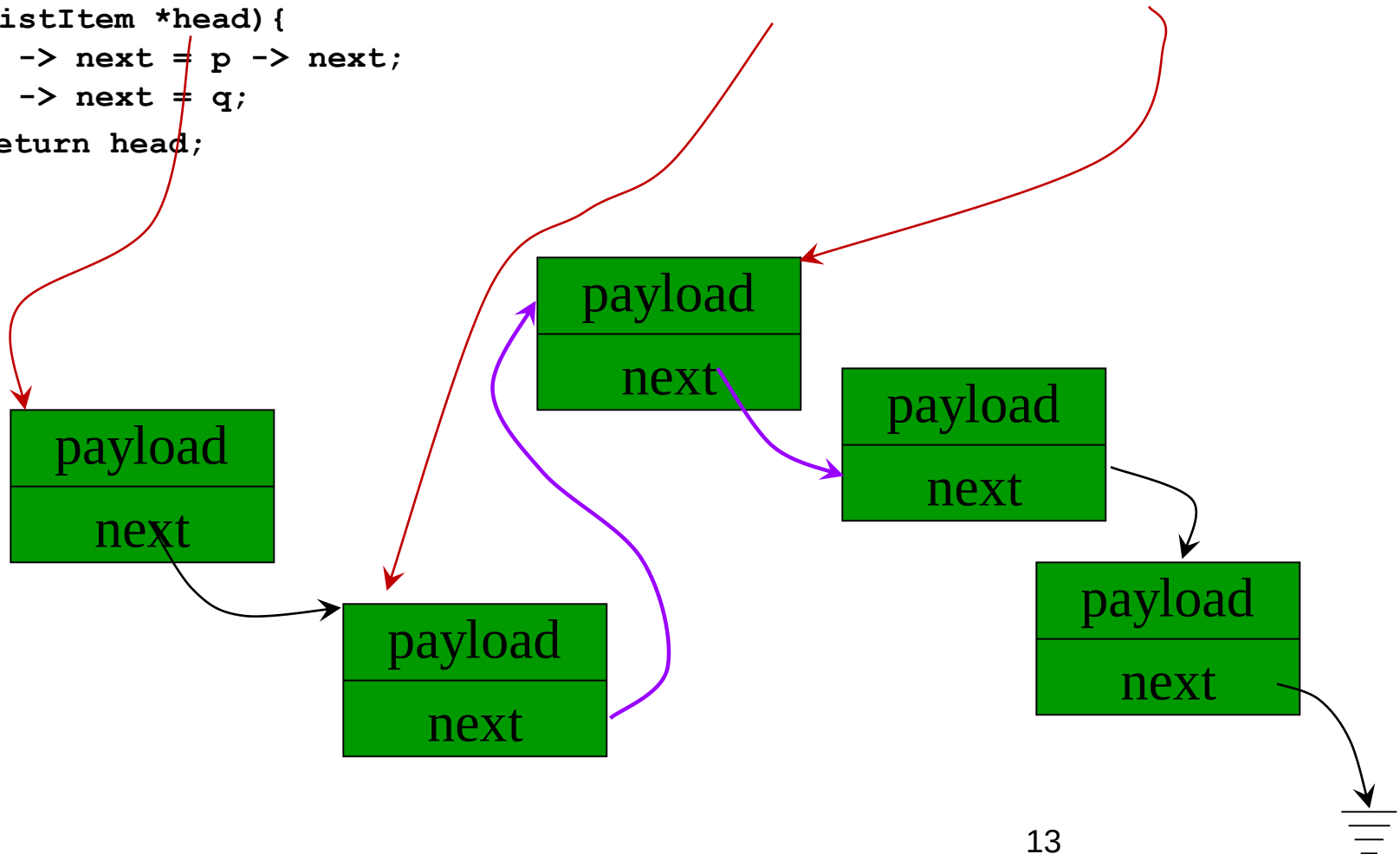
Adding an Item to a List

```
struct listItem *addAfter(struct listItem *p, struct listItem *q, struct  
    listItem *head){  
    q -> next = p -> next;  
    ?  
    return head;  
}
```



Adding an Item to a List

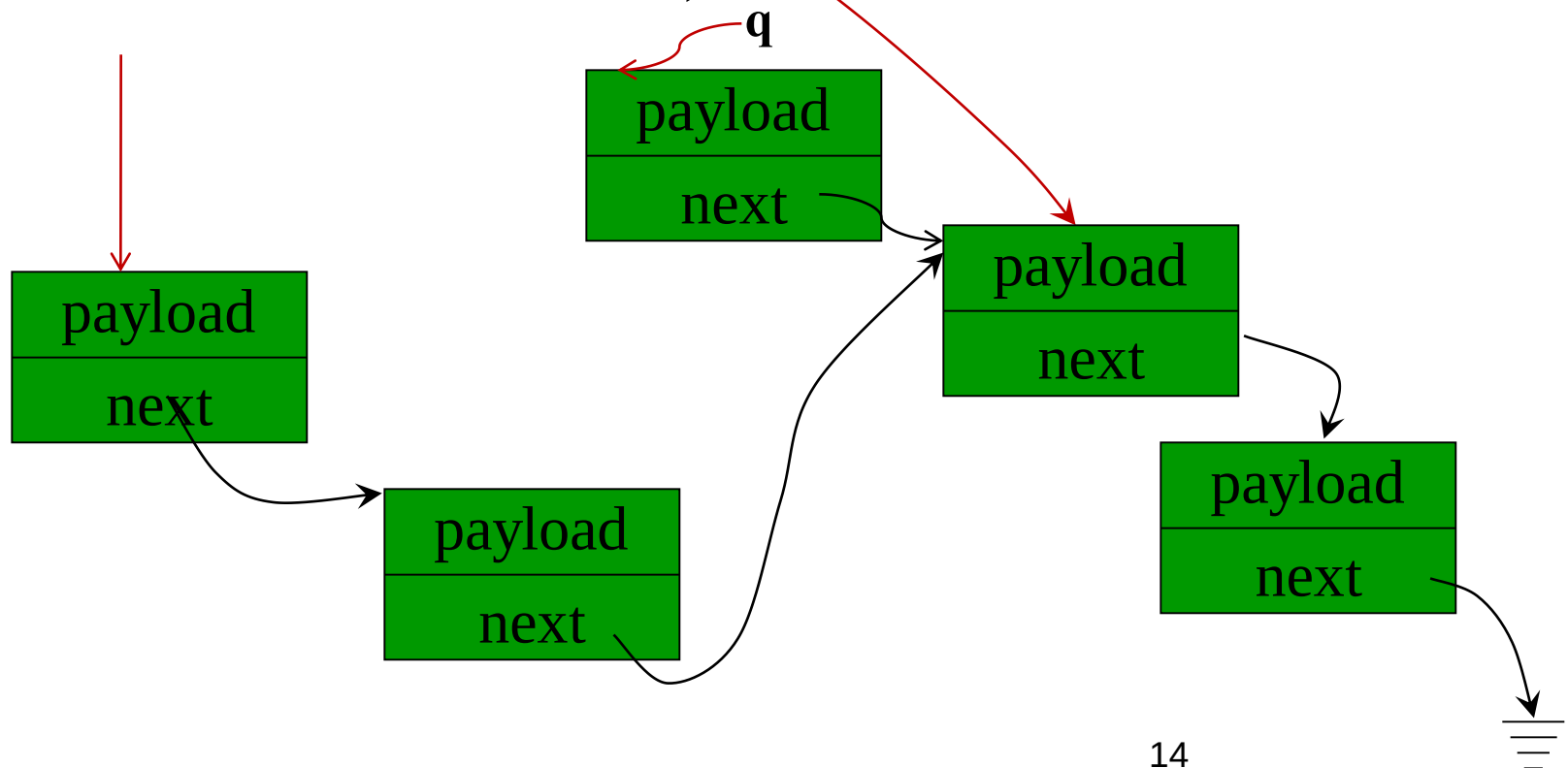
```
struct listItem *addAfter(struct listItem *p, struct listItem *q, struct  
    listItem *head){  
    q -> next = p -> next;  
    p -> next = q;  
    return head;  
}
```

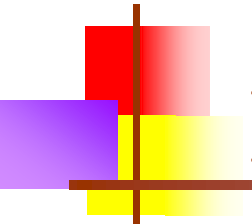


What about Adding an Item *before* another Item?

```
struct listItem *p;
```

- Add an item *before* item pointed to by **p** (**p** **!=** **NULL**)



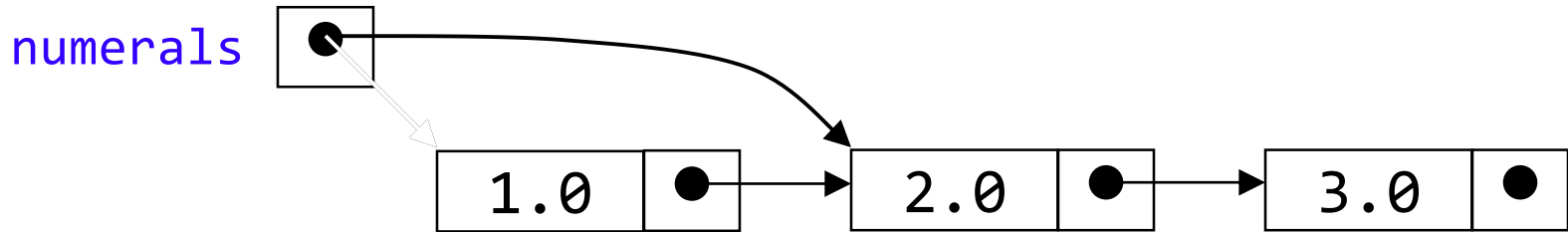


Deleting a node from a SLL

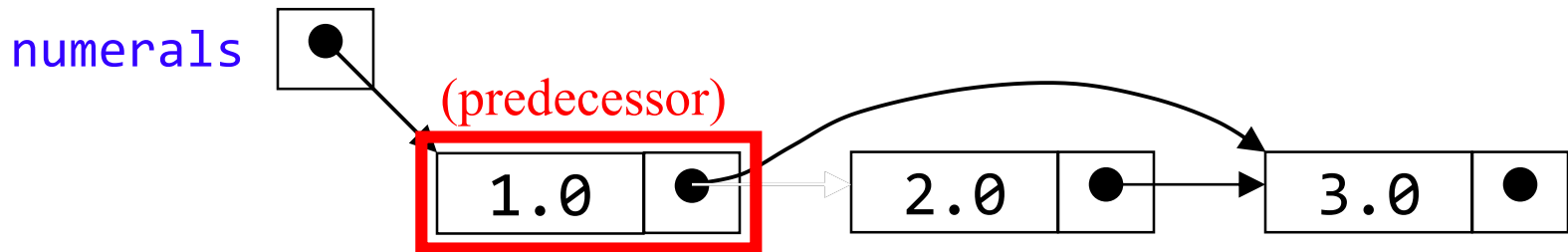
- In order to delete a node from a SLL, you have to change the link in its *predecessor*
- This is slightly tricky, because you can't follow a pointer backwards
- Deleting the first node in a list is a special case, because the node's predecessor may or may not be the list header.

Deleting an element from a SLL

- To delete the first element of a non-empty list, change the link in the header (if exists) or change the pointer which contains the address of the first node in the list.



- To delete some other element, change the link in its predecessor

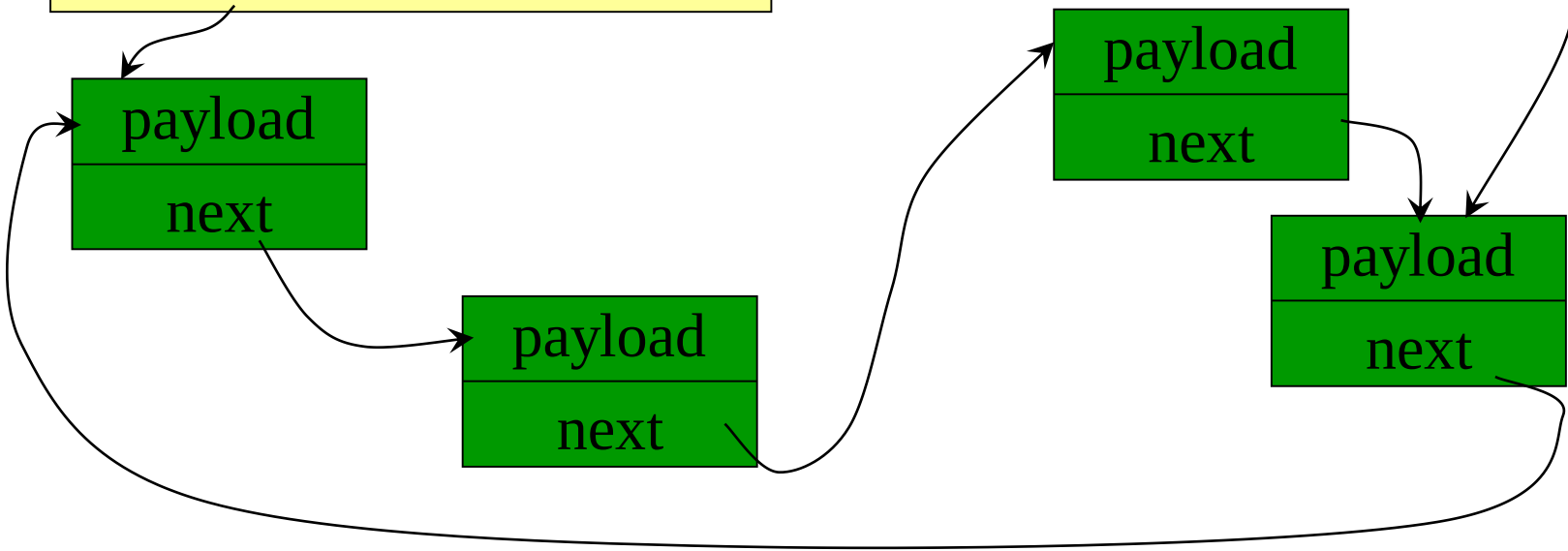


Circular List

Optional:—

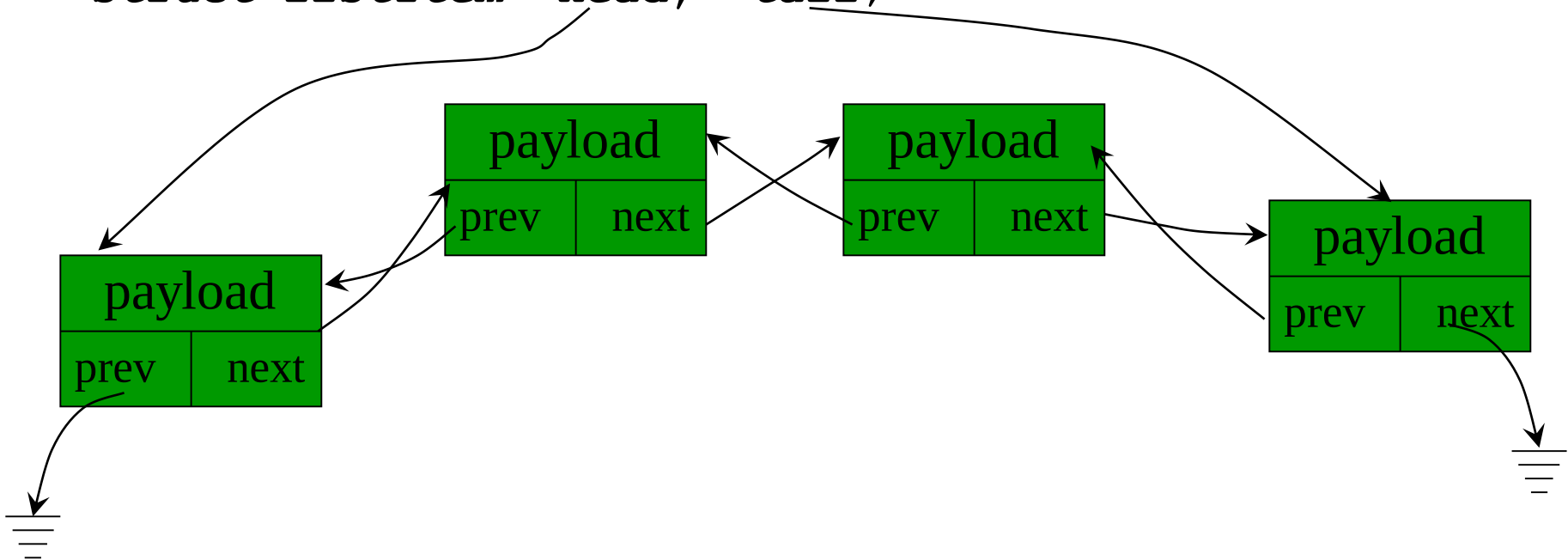
```
struct listItem *head;
```

```
struct listItem *tail;
```



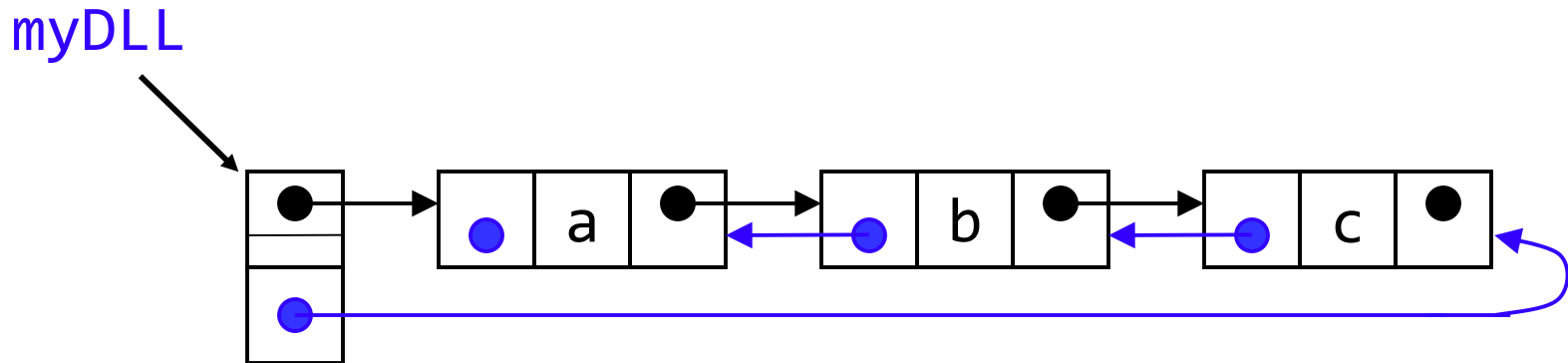
Doubly-Linked List

```
struct listItem {  
    type payload;  
    struct listItem *prev;  
    struct listItem *next;  
};  
struct listItem *head, *tail;
```

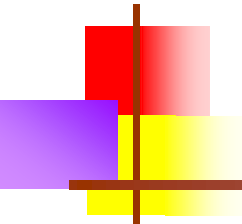


Doubly-linked lists

- Here is a **doubly-linked list (DLL)**:



- Each node contains a value, a link to its successor (if any), *and* a link to its predecessor (if any)
- The header's links points to the first node in the list *and* to the last node in the list (or contains null links if the list is empty)



DLLs compared to SLLs

■ Advantages:

- Can be traversed in either direction (may be essential for some programs)
- Some operations, such as deletion and inserting before a node, become easier

■ Disadvantages:

- Requires more space
- List manipulations are slower (because more links must be changed)
- Greater chance of having bugs (because more links must be manipulated)

Deleting a node from a DLL

- Node deletion from a DLL involves changing *two* links
- In this example, we will delete node b

